30-Day PostgreSQL Course Session Recording Script

Day 4 — Logical Operators

AND, OR, NOT, IN, BETWEEN & Operator Precedence

Field	Detail
Estimated Duration	35–40 minutes
Level	Beginner (Week 1 — SQL Foundations)
Session Goal	Combine multiple filter conditions to build precise pipeline queries
Prerequisites	Days 1–3 (PostgreSQL basics, SELECT, WHERE, comparisons)
Tools	Neon PostgreSQL 16, Jupyter + ipython-sql, psycopg2

What you will learn: • Combine conditions with AND, OR, and NOT • Use IN as a cleaner alternative to chained ORs • Apply BETWEEN for inclusive numeric and date ranges • Understand and prevent operator precedence bugs • Build a pipeline quality-gate filter using all operators

Session Agenda

Segment	Topic	Duration
00:00	Intro & recap of Day 3	3 min
03:00	AND operator	5 min
08:00	OR operator	4 min
12:00	NOT operator	3 min
15:00	IN and NOT IN	5 min
20:00	BETWEEN	4 min
24:00	Operator precedence — the gotcha	5 min
29:00	Hands-on exercise	6 min
35:00	Recap + Day 5 preview	3 min

[00:00 — 03:00] Intro & Recap of Day 3

Camera: Welcome back to the 30-Day PostgreSQL Course. Yesterday on Day 3 we learned to filter rows with the WHERE clause — single conditions using =, >, <, != and NULL checks. Today we level up: we're going to combine conditions. That means AND, OR, NOT, IN, and BETWEEN — the building blocks of real-world pipeline filters.

Quick Day 3 recap:

```
SELECT * FROM orders WHERE amount > 100; -- simple comparison
SELECT * FROM orders WHERE status != 'shipped'; -- not equal
SELECT * FROM orders WHERE notes IS NULL; -- null check
```

Camera: Good. Today we're layering these together. Let's set up our table first.

```
-- Setup: run this in your notebook
DROP TABLE IF EXISTS pipeline_events;
CREATE TABLE pipeline_events (
event_id SERIAL PRIMARY KEY,
source TEXT,
status TEXT,
amount NUMERIC(10,2),
region TEXT,
loaded_at TIMESTAMP DEFAULT NOW()
);
INSERT INTO pipeline_events (source, status, amount, region) VALUES
('api','completed',250.00,'US'), ('webhook','failed',15.50,'EU'),
('api','completed',480.00,'US'), ('file','completed',8.00,'APAC'),
('webhook','error',95.00,'EU'), ('api','pending',120.00,'US'), ...
```

[03:00 — 08:00] AND — Both Must Be True

Camera: AND is the strictest operator. Every condition joined by AND must be true for a row to pass. In pipelines, AND is how you load records that are both from the right source AND in the right state.

```
-- Only API events that completed successfully
SELECT event_id, source, status, amount
FROM pipeline_events
WHERE source = 'api'
AND status = 'completed';
```

Talking point: AND is like a checklist — the row must pass every item. Failing any one condition removes the row from results. This is how pipeline quality gates work: records must be valid on all dimensions.

> **Pipeline tip: In ETL/ELT pipelines, the very first step after ingestion is usually a quality gate. AND chains let you define exactly which records are 'clean enough' to promote to the next layer.**

[08:00 — 12:00] OR — Either Will Do

Camera: OR is more permissive. If any one of the OR conditions is true, the row is included. Use OR to capture multiple valid states at once — like all the ways a record can fail.

```
-- Capture all problem records in one pass
SELECT event_id, source, status, amount
FROM pipeline_events
WHERE status = 'failed'
OR status = 'error';
```

Talking point: In monitoring and alerting, you almost never care which specific type of failure occurred — you want all of them. OR lets you express that cleanly.

[12:00 — 15:00] NOT — Flipping the Result

Camera: NOT inverts any condition. It's useful when it's easier to define what you DON'T want than what you do.

```
-- Everything that isn't completed
SELECT event_id, source, status, amount
FROM pipeline_events
WHERE NOT status = 'completed';

-- Equivalent shorthand:
WHERE status != 'completed'
```

Talking point: NOT shines when combined with IS NULL — `WHERE NOT amount IS NULL` is the same as `WHERE amount IS NOT NULL`, but the NOT form reads naturally in complex expressions.

[15:00 — 20:00] IN and NOT IN — Lists Made Easy

Camera: IN is a shortcut for multiple OR conditions on the same column. Instead of a long OR chain, you write a comma-separated list. NOT IN is its inverse — exclude a list.

```
-- Without IN (messy):
WHERE status = 'failed' OR status = 'error' OR status = 'pending'

-- With IN (clean):
WHERE status IN ('failed', 'error', 'pending')

-- NOT IN — denylist pattern:
WHERE status NOT IN ('error', 'failed')
```

Pipeline pattern — Allowlists and denylists: • Allowlist: WHERE source IN ('api', 'webhook') — only trusted sources pass • Denylist: WHERE status NOT IN ('error','failed') — block known bad states Both patterns are everywhere in pipeline routing and quality filtering.

Talking point: IN is also much easier to maintain. When a new source is added to the trusted list, you just append a value inside the parentheses.

[20:00 — 24:00] BETWEEN — Inclusive Range Checks

Camera: BETWEEN is shorthand for a range check. It's inclusive — both the lower and upper bound are included in the results.

```
-- Medium-value events
SELECT event_id, source, amount
FROM pipeline_events
WHERE amount BETWEEN 50 AND 300;

-- Equivalent without BETWEEN:
WHERE amount >= 50 AND amount <= 300
```

Talking point: BETWEEN works on numbers, dates, and timestamps. For date ranges in pipelines — say, 'load records from last week' — BETWEEN is extremely readable.

```
-- Date range example (preview of Day 7):
WHERE event_date BETWEEN '2024-01-01' AND '2024-01-31'
```

[24:00 — 29:00] ■■ Operator Precedence — The Gotcha

Camera: This is the most important slide of the day. AND has higher precedence than OR — it's evaluated first, just like multiplication before addition in math. This causes silent bugs that are very hard to spot.

```
-- WRONG — NOT what you probably intended:
WHERE region = 'EU'
OR region = 'US'
AND status = 'completed';

-- SQL reads this as:
-- region = 'EU' OR (region = 'US' AND status = 'completed')
-- EU rows appear regardless of status!
```

```
-- CORRECT — parentheses make intent explicit:
WHERE (region = 'EU' OR region = 'US')
AND status = 'completed';

-- Now reads as: (either region) AND completed
```

Rule of thumb: Whenever you mix AND and OR in the same WHERE clause, ALWAYS add parentheses. Even if you know the precedence rules, parentheses make your intent clear to the next person who reads the query — including future you.

Common Errors

Common Mistake	What Actually Happens	Fix
Mixing OR and AND without parentheses	AND binds first — EU rows ignore the status filter	Wrap OR groups in ()
BETWEEN with wrong order	BETWEEN 300 AND 50 returns nothing	Always low AND high
NOT IN with a NULL in the list	Returns no rows — NULL poisons NOT IN	Use IS NOT NULL alongside

[29:00 — 35:00] Hands-On Exercise

Camera: Pause the video. Open your Day 4 notebook and complete the exercise below.

> **Exercise: Pipeline quality gate Write a query on pipeline_events that returns events where: 1. source is 'api' or 'webhook' 2. AND status is NOT 'error' 3. AND amount is between 10 and 500 Return: event_id, source, status, amount**

Solution:

```
SELECT
event_id,
source,
status,
amount
FROM pipeline_events
WHERE source IN ('api', 'webhook')
AND status != 'error'
AND amount BETWEEN 10 AND 500;
```

Expected: You should see api and webhook rows with amounts from 10–500, with no error rows included.

[35:00 — 38:00] Recap & Day 5 Preview

What you learned today:

Operator	Purpose	Pipeline Use Case
AND	All conditions must be true	Quality gates, multi-dimension filters
OR	Any condition can be true	Capture all failure modes
NOT	Invert a condition	Exclusion patterns
IN (...)	Match any value in a list	Allowlists for trusted sources
NOT IN (...)	Exclude a list of values	Denylists for bad statuses
BETWEEN a AND b	Inclusive range	Amount bins, date windows
()	Force evaluation order	Prevent precedence bugs

Camera: Tomorrow is Day 5 — ORDER BY, LIMIT, and OFFSET. You'll learn how to sort your results and grab only the top N rows. This is essential for pagination and 'latest record' patterns in pipelines. See you then!

Appendix — Quick Reference Card

```
-- AND: both conditions must be true
WHERE source = 'api' AND status = 'completed'

-- OR: either condition true
WHERE status = 'failed' OR status = 'error'

-- NOT: invert
WHERE NOT status = 'completed'

-- IN: match a list
WHERE source IN ('api', 'webhook')

-- NOT IN: exclude a list
WHERE status NOT IN ('error', 'failed')

-- BETWEEN: inclusive range
WHERE amount BETWEEN 10 AND 500

-- Precedence: AND before OR — always use parentheses!
WHERE (region = 'EU' OR region = 'US') AND status = 'completed'
```

Key rule: AND is evaluated before OR. Parentheses override precedence and document intent. When in doubt, add parentheses.